Machine Learning II - Examen Parcial I

September 30, 2026



Examinador: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub * URL: https://github.com/Exosphere-Idyllic/Machine-Learning

0.1 Examen Parcial I

0.2 Visión general del pipeline

Objetivo del proyecto: construir un modelo de **regresión lineal múltiple** que estime el costo del seguro médico (`charges`) de una persona a partir de sus características (edad, sexo, IMC, hijos, fumador y región), usando el dataset *Medical Cost Personal Datasets* de Kaggle (`mirichoi0218/insurance`).

Un **pipeline** es la secuencia ordenada de pasos que lleva los datos crudos hasta una predicción. Cada paso de este notebook recibe la salida del anterior:

Paso	Fase	Qué hace	Resultado que produce
1	Configuración inicial	Importa librerías y fija la semilla aleatoria	Entorno listo y reproducible
2	Descarga del dataset	Baja el CSV desde Kaggle y lo carga en un DataFrame	`df` (1338 × 7)
3	Comprensión de los datos	Revisa tipos, estadísticas, nulos y duplicados	Diagnóstico de calidad

Paso	Fase	Qué hace	Resultado que produce
4	EDA	Explora distribuciones, categorías y correlaciones	Hallazgos para el modelado
5	Detección de outliers	Aplica los criterios IQR y Z-score	Conteo de valores atípicos
6	Preparación y estandarización	Separa train/test, escala y codifica variables	`X_train_p`, `X_test_p`, `preprocessor`
7	Selección de atributos	Compara 4 técnicas de importancia de variables	Tabla `sel`
8	Validación	Validación cruzada de 5 particiones	R^2 y RMSE de CV
9	Entrenamiento	Ajusta el modelo final sobre el conjunto de entrenamiento	`pipe` entrenado y coeficientes
10	Evaluación	Mide el desempeño en datos no vistos y prueba una mejora	Métricas y gráficos de diagnóstico
11	Predicción	Estima `charges` para personas nuevas	`nuevos` con `charges_estimado`

Variable objetivo (y): `charges`. **Variables predictoras (X):** `age`, `sex`, `bmi`, `children`, `smoker`, `region`.

0.3 Paso 1 — Configuración inicial del entorno

Qué hace este paso del pipeline: prepara todas las herramientas que se usarán en el resto del proyecto (descarga, análisis exploratorio, gráficos, estadística, preprocesamiento, selección de atributos, modelo y métricas) y fija una **semilla aleatoria** para que los resultados sean reproducibles.

Variables y componentes principales

Elemento	Tipo	Descripción
`kagglehub`	librería	Descarga datasets de Kaggle desde código
`numpy` (`np`) / `pandas` (`pd`)	librerías	Cálculo numérico y manejo de tablas (DataFrames)
`matplotlib.pyplot` (`plt`)	librería	Gráficos
`scipy.stats` (`stats`)	librería	Funciones estadísticas (asimetría, Z-score)
`train_test_split`, `KFold`, `cross_val_score`	funciones/clases de sklearn	Partición de datos y validación cruzada

Elemento	Tipo	Descripción
`StandardScaler`, `OneHotEncoder`	transformadores	Escalado de numéricas y codificación de categóricas
`ColumnTransformer`, `Pipeline`, `SimpleImputer`	utilidades de sklearn	Combinan transformaciones en un flujo único y tratan nulos
`SelectKBest`, `f_regression`, `mutual_info_regression`, `RFE`, `Lasso`	selección de atributos	Filtros, wrapper y método embebido
`LinearRegression`	modelo	Regresión lineal múltiple (el modelo del proyecto)
`RANDOM_STATE`	constante (`42`)	Semilla compartida por todas las operaciones aleatorias

Proceso: se importan las librerías agrupadas por función, se define `RANDOM_STATE` y se fija la semilla global de NumPy. Algunas importaciones (`io`, `zipfile`, `MinMaxScaler`, `Ridge`) quedan disponibles pero no se usan en este notebook.

[1]:
```
##   Descarga y manejo de archivos
# kagglehub: permite descargar datasets de Kaggle directamente desde código
import kagglehub
# os: manejo de rutas de archivos y carpetas del sistema operativo
import os
# io: utilidades de entrada/salida en memoria (importado, no se usa en este␣
 ↪notebook)
import io
# zipfile: lectura de archivos comprimidos .zip (importado, no se usa en este␣
 ↪notebook)
import zipfile

##   EDA (Análisis Exploratorio de Datos)
# numpy: arreglos y operaciones numéricas; se abrevia como np
import numpy as np
# pandas: tablas (DataFrame) para cargar, limpiar y analizar datos; se abrevia␣
 ↪como pd
import pandas as pd

##   Visualización
# pyplot de matplotlib: histogramas, barras, dispersión y mapas de calor
import matplotlib.pyplot as plt

##   Estadística
# scipy.stats: asimetría (skew) y puntaje Z (zscore)
from scipy import stats

##   Preprocesamiento
# train_test_split: separa datos en entrenamiento/prueba
```

```
# cross_val_score: calcula una métrica en cada partición de la validación␣
 ↪cruzada
# KFold: define cómo se parten los datos en k bloques
from sklearn.model_selection import train_test_split, cross_val_score, KFold
# StandardScaler: estandariza (media 0, desviación 1); MinMaxScaler: escala a␣
 ↪[0,1] (no se usa aquí)
# OneHotEncoder: convierte categorías en columnas 0/1
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
# ColumnTransformer: aplica transformaciones distintas a grupos de columnas␣
 ↪distintos
from sklearn.compose import ColumnTransformer
# Pipeline: encadena pasos (preprocesamiento + modelo) en un solo objeto
from sklearn.pipeline import Pipeline
# SimpleImputer: rellena valores faltantes con una estrategia (media, mediana,␣
 ↪etc.)
from sklearn.impute import SimpleImputer

##   Selección de atributos
# SelectKBest + f_regression: filtro basado en la prueba F
# mutual_info_regression: filtro basado en información mutua
# RFE: eliminación recursiva de atributos (método wrapper)
from sklearn.feature_selection import SelectKBest, f_regression,␣
 ↪mutual_info_regression, RFE
# LinearRegression, Lasso (embebido, penalización L1), Ridge (penalización L2,␣
 ↪no se usa aquí)
from sklearn.linear_model import LinearRegression, Lasso, Ridge

##   Modelo
# Importación repetida de LinearRegression (ya está arriba; no causa ningún␣
 ↪problema)
from sklearn.linear_model import LinearRegression

##   Métricas
# MSE (error cuadrático medio), MAE (error absoluto medio), R² (varianza␣
 ↪explicada),
# MAPE (error porcentual absoluto medio)
from sklearn.metrics import mean_squared_error, mean_absolute_error,r2_score,␣
 ↪mean_absolute_percentage_error

##   Reproducibilidad
# Semilla única del proyecto: mismas divisiones y mismos resultados en cada␣
 ↪ejecución
RANDOM_STATE = 42
# Fija la semilla del generador aleatorio global de NumPy
np.random.seed(RANDOM_STATE)
```

```
# Mensaje de confirmación de que todas las importaciones funcionaron
print("Librerías cargadas correctamente")
```

```
Librerías cargadas correctamente
```

0.4 Paso 2 — Descarga y carga del dataset

Qué hace este paso del pipeline: obtiene los datos crudos. Descarga el dataset desde Kaggle mediante código (sin bajarlo a mano), localiza el archivo `insurance.csv` y lo carga en un DataFrame de pandas.

Variables

Variable	Tipo	Descripción
`path`	`str`	Carpeta local donde `kagglehub` guarda (o ya tiene en caché) el dataset
`csv_path`	`str`	Ruta completa al archivo `insurance.csv` dentro de `path`
`df`	`DataFrame`	Tabla con los datos originales: 1338 filas × 7 columnas

Columnas del dataset

Columna	Tipo	Significado
`age`	numérica discreta	Edad del asegurado
`sex`	categórica	Sexo (`female` / `male`)
`bmi`	numérica continua	Índice de masa corporal
`children`	numérica discreta	Número de hijos cubiertos por el seguro
`smoker`	categórica	¿Fuma? (`yes` / `no`)
`region`	categórica	Región residencial en EE. UU. (`northeast`, `northwest`, `southeast`, `southwest`)
`charges`	numérica continua	**Variable objetivo:** costo médico facturado por el seguro

Proceso: `dataset_download` descarga y devuelve la ruta → `os.path.join` arma la ruta del CSV → `pd.read_csv` lo lee → se imprime `df.shape` y se muestran las primeras 5 filas con `df.head()` como verificación visual.

[2]:
```
# Descarga el dataset "mirichoi0218/insurance" de Kaggle y devuelve la carpeta␣
↪local donde quedó
path = kagglehub.dataset_download("mirichoi0218/insurance")
# Muestra la ruta de descarga para dejar constancia de dónde están los archivos
print("Path to dataset files:", path)
```

```
# Localizar el CSV dentro del directorio descargado
# os.path.join arma la ruta usando el separador correcto del sistema operativo
csv_path = os.path.join(path, "insurance.csv")
# Lee el archivo CSV y lo guarda como DataFrame (cada columna del CSV pasa a ser una columna de df)
df = pd.read_csv(csv_path)

# Muestra las dimensiones como (filas, columnas); se espera (1338, 7)
print(f"Dimensiones: {df.shape}")
# Última expresión de la celda: Jupyter muestra automáticamente las primeras 5 filas
df.head()
```

```
Path to dataset files:
C:\Users\USER\.cache\kagglehub\datasets\mirichoi0218\insurance\versions\1
Dimensiones: (1338, 7)
```

[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

0.5 Paso 3 — Carga y comprensión de los datos (estructura y calidad)

Qué hace este paso del pipeline: antes de transformar nada, se inspecciona el DataFrame para conocer su estructura y detectar problemas de calidad (nulos y duplicados). Es la base para decidir qué limpieza y qué preprocesamiento se necesitan.

Qué hace cada instrucción de la celda

Instrucción	Qué responde
`df.info()`	Número de filas, tipo de dato de cada columna, valores no nulos y uso de memoria
`df.describe(include="all")`	Estadísticos de las numéricas (media, desviación, mínimo, cuartiles, máximo) y de las categóricas (categorías únicas, la más frecuente y su frecuencia)
`df.duplicated().sum()`	Cuántas filas están completamente repetidas
`df.isnull().sum()`	Cuántos valores faltantes hay por columna

0.5.1 Estructura del dataset

- **Dimensiones:** 1338 registros (filas) y 7 columnas.
- **Tipos de datos:**
  - `int64`: `age`, `children` (numéricas discretas).

  - `float64`: `bmi`, `charges` (numéricas continuas).
  - `object/str`: `sex`, `smoker`, `region` (categóricas).
- **Memoria:** 73.3 KB, un dataset ligero que se procesa en memoria sin problemas.
- **Valores no nulos:** todas las columnas tienen 1338 valores no nulos, así que no hay valores faltantes (se confirma con `isnull().sum()`).
- **Duplicados:** se detecta **1** fila duplicada; se elimina en el Paso 4.

0.5.2 Observaciones clave

- **`charges` (variable objetivo):** media 13,270 y desviación estándar muy alta (12,110), con máximo de 63,770. Esto sugiere una distribución **asimétrica (sesgada a la derecha)**, típica de costos médicos. Se revisa en el EDA.
- **`bmi`:** la media (30.66) está en el rango de sobrepeso/obesidad según la OMS. El máximo (53.13) indica posibles **valores atípicos** que se analizan en el Paso 5.
- **Variables categóricas:** `sex`, `smoker` y `region` tienen pocas categorías (2, 2 y 4), por lo que son candidatas ideales para **One-Hot Encoding**.
- **`age`:** rango de 18 a 64 años, sin valores extraños.

[3]:
```
# Resumen estructural: filas, columnas, tipo de dato, conteo de no nulos y␣
 ↪memoria usada
df.info()
# Estadísticas descriptivas de TODAS las columnas (numéricas y categóricas);␣
 ↪display() fuerza a mostrar la tabla
display(df.describe(include="all"))
# Cuenta filas completamente duplicadas (True se suma como 1)
print("Duplicados:", df.duplicated().sum())
# Cuenta valores nulos por columna; "\n" baja de línea para que la tabla quede␣
 ↪legible
print("Nulos:\n", df.isnull().sum())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   str
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   str
 5   region    1338 non-null   str
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), str(3)
memory usage: 73.3 KB
```

	age	sex	bmi	children	smoker	region
count	1338.000000	1338	1338.000000	1338.000000	1338	1338
unique	NaN	2	NaN	NaN	2	4
top	NaN	male	NaN	NaN	no	southeast

```
freq            NaN   676          NaN          NaN   1064        364
mean      39.207025   NaN    30.663397     1.094918    NaN        NaN
std       14.049960   NaN     6.098187     1.205493    NaN        NaN
min       18.000000   NaN    15.960000     0.000000    NaN        NaN
25%       27.000000   NaN    26.296250     0.000000    NaN        NaN
50%       39.000000   NaN    30.400000     1.000000    NaN        NaN
75%       51.000000   NaN    34.693750     2.000000    NaN        NaN
max       64.000000   NaN    53.130000     5.000000    NaN        NaN
```

```
             charges
count    1338.000000
unique           NaN
top              NaN
freq             NaN
mean    13270.422265
std     12110.011237
min      1121.873900
25%      4740.287150
50%      9382.033000
75%     16639.912515
max     63770.428010
```

```
Duplicados: 1
Nulos:
 age         0
sex          0
bmi          0
children     0
smoker       0
region       0
charges      0
dtype: int64
```

0.6 Paso 4 — Limpieza básica y Análisis Exploratorio de Datos (EDA)

Qué hace este paso del pipeline: elimina el registro duplicado y explora visualmente cómo se distribuyen las variables y cómo se relacionan con `charges`. El objetivo es entender los datos para justificar las decisiones de modelado.

Variables

Variable	Tipo	Descripción
`df`	`DataFrame`	Se reasigna **sin duplicados** y con el índice reiniciado (1337 filas)
`num_cols`	`list[str]`	Columnas numéricas: `age`, `bmi`, `children`, `charges`
`cat_cols`	`list[str]`	Columnas categóricas: `sex`, `smoker`, `region`

Variable	Tipo	Descripción
`fig`, `ax`	`Figure`, arreglo de `Axes`	Lienzo y los 3 ejes de la figura de barras
`a`, `c`	`Axes`, `str`	Variables del bucle: eje actual y categoría que se grafica
`corr`	`DataFrame`	Matriz de correlación de Pearson entre las variables numéricas

Procesos y qué muestra cada gráfico

1. **`drop_duplicates` + `reset_index`:** quita la fila repetida y renumera el índice de 0 a n-1.
2. **Histogramas** de las 4 numéricas: muestran la forma de cada distribución.
3. **Asimetría (`skew`) de `charges`:** el resultado es **1.51**; un valor positivo y mayor que 1 confirma un fuerte sesgo a la derecha (pocos cargos muy altos). Esta es la razón por la que más adelante se prueba una transformación logarítmica.
4. **Barras de `charges` medio por categoría:** compara el costo promedio entre sexos, fumadores/no fumadores y regiones.
5. **Mapa de calor de correlaciones:** resume la relación lineal entre pares de variables numéricas (rojo = positiva, azul = negativa).
6. **Dispersión `bmi` vs `charges` coloreada por `smoker`:** rojo = fumador, azul = no fumador. Permite comprobar visualmente que **fumar** cambia fuertemente el nivel de costos y modifica la relación entre IMC y costo (este es el hallazgo clave del EDA, que luego confirman los coeficientes del modelo).

[4]:
```
# Elimina filas duplicadas y reinicia el índice (drop=True evita guardar el␣
 ↪índice viejo como columna)
df = df.drop_duplicates().reset_index(drop=True)   # hay 1 duplicado en este␣
 ↪dataset

# Lista de columnas numéricas (incluye la variable objetivo charges)
num_cols = ["age", "bmi", "children", "charges"]
# Lista de columnas categóricas
cat_cols = ["sex", "smoker", "region"]

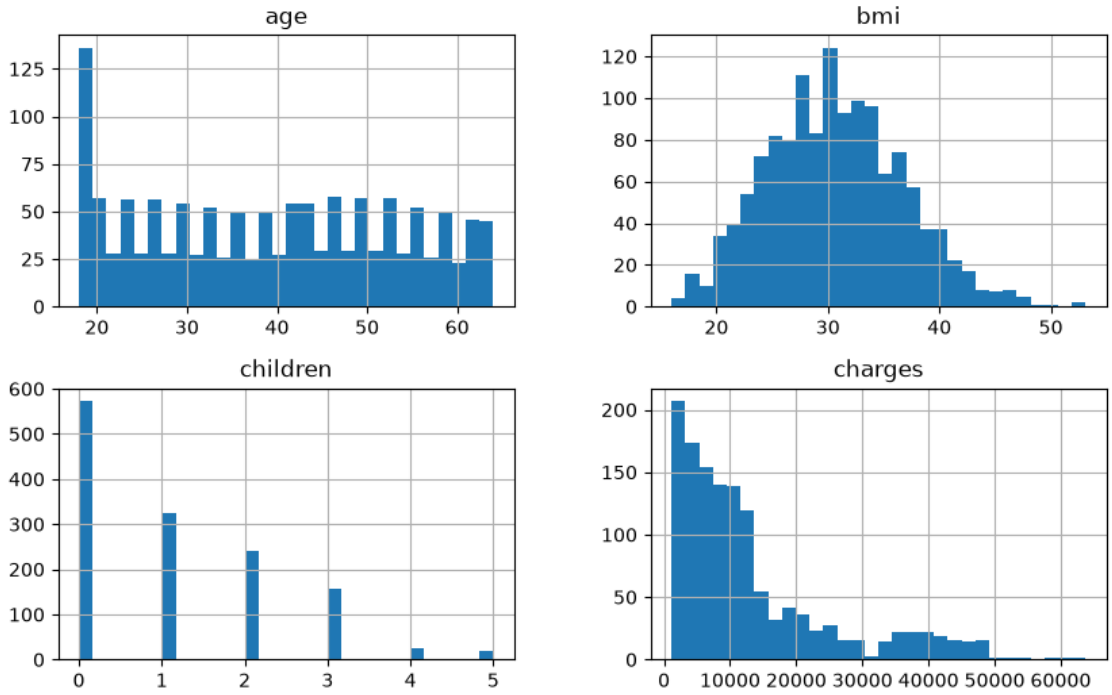
# Dibuja un histograma por cada columna numérica (30 intervalos) en una figura␣
 ↪de 10x6 pulgadas
df[num_cols].hist(bins=30, figsize=(10, 6))
# Muestra la figura en pantalla
plt.show()
# Calcula la asimetría de charges (>0 indica cola larga a la derecha)
print("Asimetría de charges:", stats.skew(df["charges"]))

# charges promedio por categoría
# Crea una figura con 1 fila y 3 gráficos, uno por variable categórica
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
```

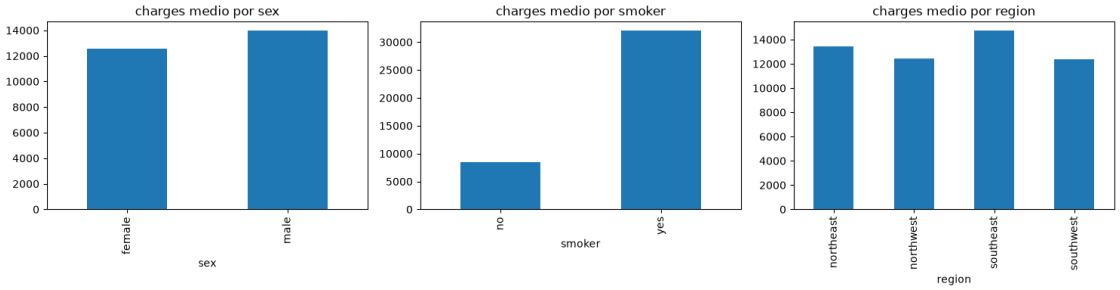
```
# Recorre cada eje junto con su variable categórica correspondiente
for a, c in zip(ax, cat_cols):
    # Agrupa por la categoría, promedia charges y lo dibuja como barras en el␣
 ↪eje a
    df.groupby(c)["charges"].mean().plot(kind="bar", ax=a, title=f"charges␣
 ↪medio por {c}")
# Ajusta márgenes para que los títulos no se encimen
plt.tight_layout()
# Muestra la figura de barras
plt.show()

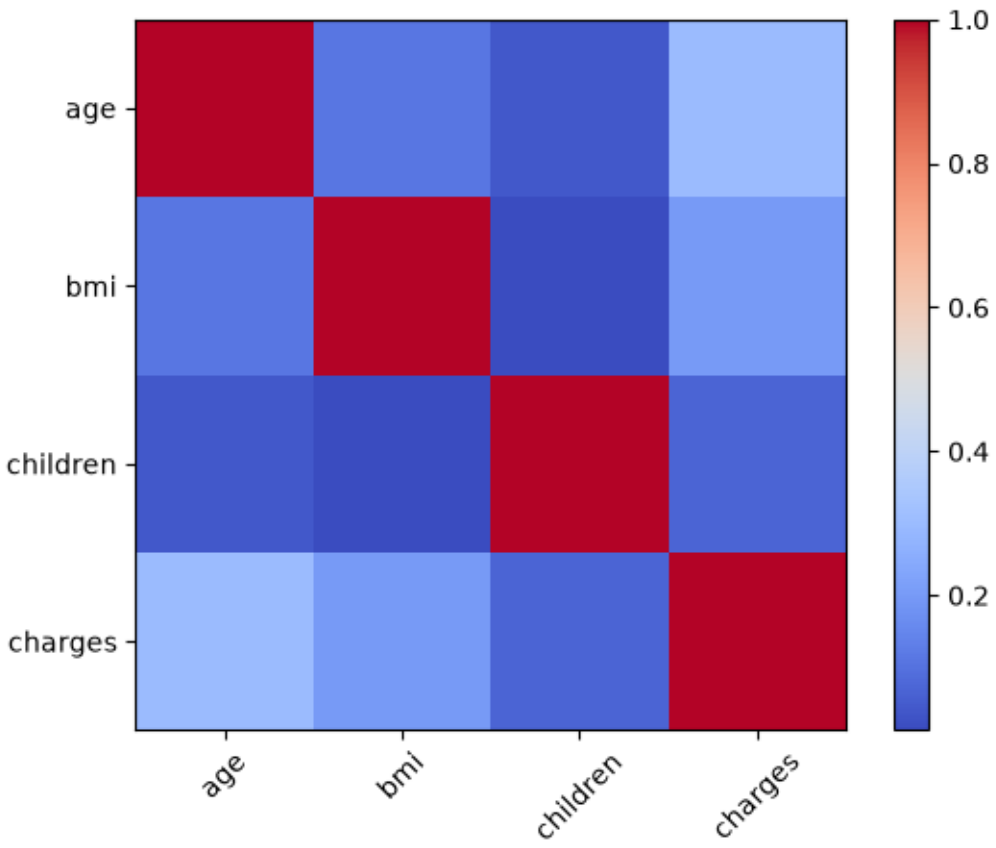
# correlaciones
# Matriz de correlación de Pearson entre las variables numéricas
corr = df[num_cols].corr()
# Dibuja la matriz como mapa de calor (coolwarm: azul=negativa, rojo=positiva)
plt.imshow(corr, cmap="coolwarm")
# Agrega la barra de color con la escala de correlación
plt.colorbar()
# Etiqueta el eje X con los nombres de las variables (rotados 45° para que se␣
 ↪lean)
plt.xticks(range(len(num_cols)), num_cols, rotation=45)
# Etiqueta el eje Y con los nombres de las variables
plt.yticks(range(len(num_cols)), num_cols)
# Muestra el mapa de calor
plt.show()

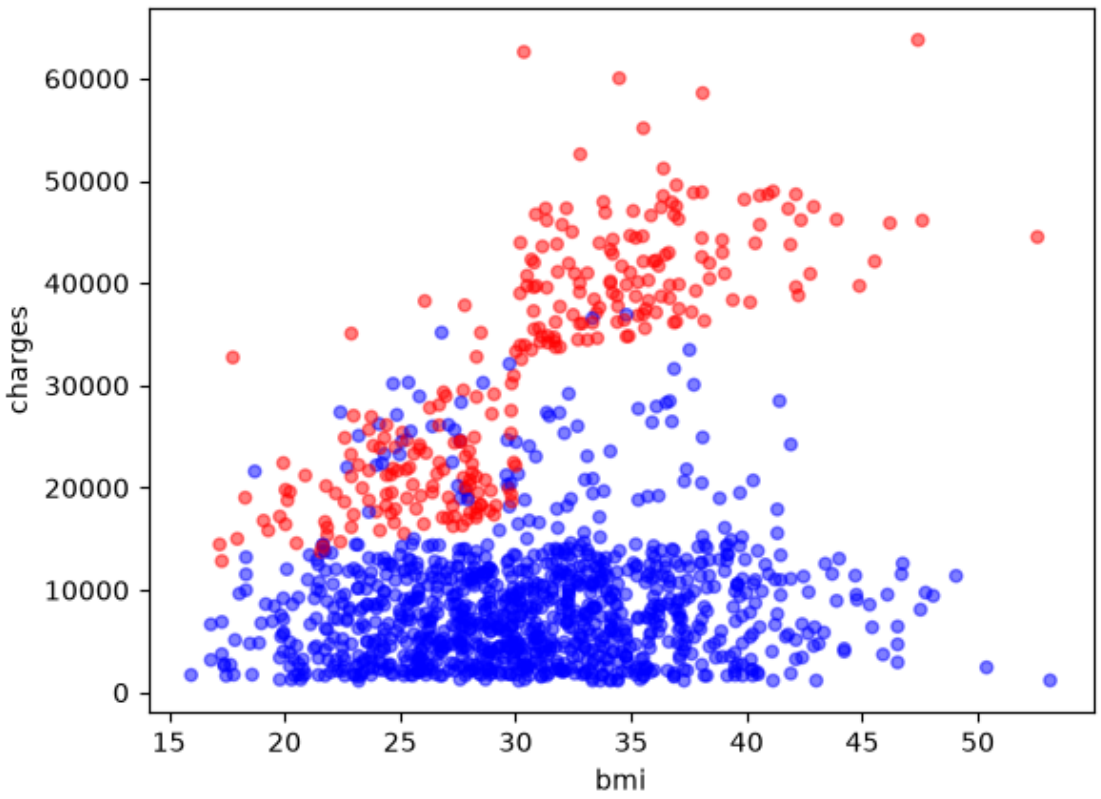
# el hallazgo clave: smoker vs bmi
# Dispersión bmi (X) vs charges (Y); el color depende de si la persona fuma:
#   (df["smoker"] == "yes") crea True/False -> .map lo convierte a "red"/"blue";
 ↪ alpha=.5 da transparencia
df.plot.scatter(x="bmi", y="charges", c=(df["smoker"] == "yes").map({True:␣
 ↪"red", False: "blue"}), alpha=.5)
# Muestra el gráfico de dispersión
plt.show()
```




```
Asimetría de charges: 1.5136902413349014
```




age
bmi
children
charges
age
bmi
children
charges
1.0
0.8
0.6
0.4
0.2



0.7 Paso 5 — Detección de valores atípicos (outliers)

Qué hace este paso del pipeline: identifica observaciones extremas con dos criterios estadísticos distintos y las visualiza con diagramas de caja. **Solo se detectan y se cuantifican; no se eliminan**, por lo que el DataFrame `df` no cambia en este paso.

Variables y funciones

Elemento	Descripción
`iqr_mask(s)`	Función que recibe una serie `s` y devuelve una máscara booleana: `True` donde el valor es atípico según el criterio IQR
`q1`, `q3`	Primer y tercer cuartil (percentiles 25 y 75)
`iqr`	Rango intercuartílico: `q3 - q1`
`c`	Variable del bucle: nombre de la columna que se evalúa
`z`	Valor absoluto del puntaje Z de `bmi` y `charges`

Criterios utilizados

- **IQR (Tukey):** es atípico todo valor menor que `Q1 − 1.5·IQR` o mayor que `Q3 + 1.5·IQR`. No supone normalidad, por eso funciona bien con distribuciones sesgadas.

- **Z-score:** mide a cuántas desviaciones estándar de la media está un valor; se considera atípico si `|Z| > 3`. Supone una distribución aproximadamente simétrica.

Resultados obtenidos

Variable	Outliers (IQR)	Outliers ($Z > 3$)
`age`	0	—
`bmi`	9	4
`children`	0	—
`charges`	139	7

`charges` tiene muchos más atípicos con IQR que con Z porque su distribución está sesgada a la derecha: la cola larga infla la desviación estándar y el criterio Z se vuelve más permisivo. Estos valores se conservan porque corresponden a costos médicos reales, no a errores de captura.

[5]:
```
# Define una función que marca qué valores de una serie son atípicos según el␣
 ↪criterio IQR
def iqr_mask(s):
    # Calcula Q1 (25%) y Q3 (75%) de una sola vez y los desempaqueta en dos␣
 ↪variables
    q1, q3 = s.quantile([.25, .75])
    # Rango intercuartílico: ancho de la caja central de los datos
    iqr = q3 - q1
    # Devuelve True si el valor está por debajo del límite inferior O por␣
 ↪encima del superior
    return (s < q1 - 1.5*iqr) | (s > q3 + 1.5*iqr)

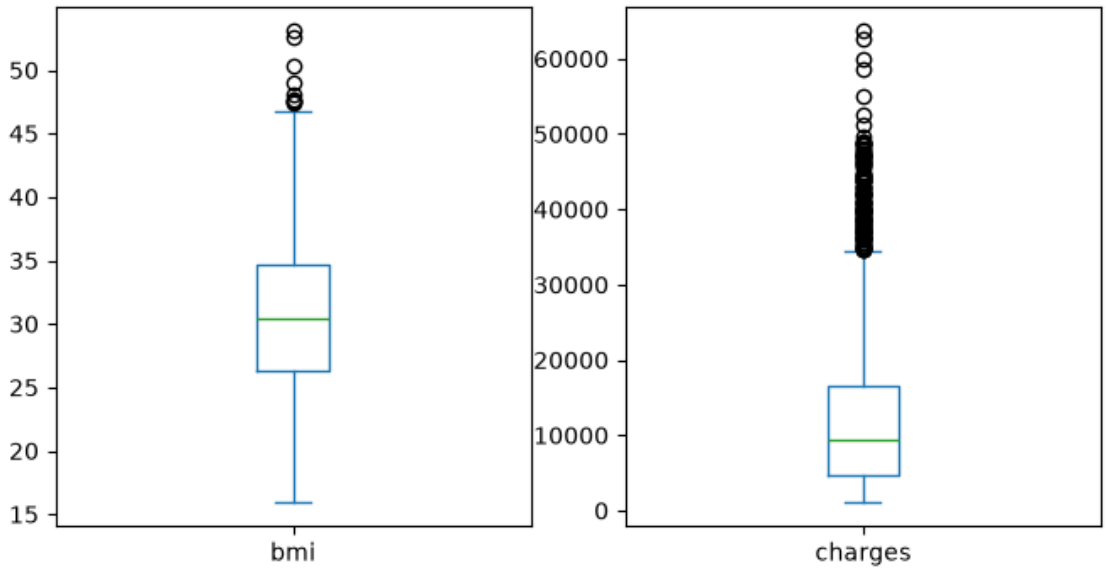
# Recorre cada columna numérica y reporta cuántos outliers tiene
for c in ["age", "bmi", "children", "charges"]:
    # iqr_mask(df[c]).sum() cuenta los True (cada True suma 1)
    print(f"{c}: {iqr_mask(df[c]).sum()} outliers (IQR)")

# Puntaje Z (en valor absoluto) de bmi y charges: distancia a la media en␣
 ↪desviaciones estándar
z = np.abs(stats.zscore(df[["bmi", "charges"]]))
# (z > 3) marca los atípicos; .sum(axis=0) cuenta por columna; se convierte a␣
 ↪dict para imprimirlo claro
print("Outliers Z>3:", pd.Series((z > 3).sum(axis=0), index=["bmi", "charges"]).
 ↪to_dict())

# Diagramas de caja de bmi y charges, uno al lado del otro (subplots=True crea␣
 ↪un gráfico por columna)
df[["bmi", "charges"]].plot(kind="box", subplots=True, figsize=(8, 4))
# Muestra los diagramas de caja
plt.show()
```

```
age: 0 outliers (IQR)
```

```
bmi: 9 outliers (IQR)
children: 0 outliers (IQR)
charges: 139 outliers (IQR)
Outliers Z>3: {'bmi': 4, 'charges': 7}
```



0.8 Paso 6 — Preparación de variables, partición y estandarización

Qué hace este paso del pipeline: separa predictoras y objetivo, divide los datos en entrenamiento y prueba, y construye el **preprocesador** que escala las variables numéricas y codifica las categóricas.

Variables

Variable	Tipo	Descripción
`X`	`DataFrame`	Todas las columnas excepto `charges` (variables predictoras)
`y`	`Series`	Columna `charges` (variable objetivo)
`X_train`, `X_test`	`DataFrame`	Predictoras de entrenamiento (80 %, 1069 filas) y de prueba (20 %, 268 filas)
`y_train`, `y_test`	`Series`	Objetivo correspondiente a cada partición
`num_f`	`list[str]`	Numéricas a escalar: `age`, `bmi`, `children`
`bin_f`	`list[str]`	Categóricas binarias: `sex`, `smoker`

Variable	Tipo	Descripción
`cat_f`	`list[str]`	Categórica de varias categorías: `region`
`preprocessor`	`ColumnTransformer`	Objeto que aplica la transformación adecuada a cada grupo de columnas
`X_train_p`, `X_test_p`	arreglos NumPy	Datos ya transformados (entrenamiento y prueba)
`feat_names`	arreglo de `str`	Nombres de las 8 columnas resultantes tras la transformación
`X_train_df`	`DataFrame`	`X_train_p` con nombres de columna, para inspeccionarlo

Procesos

1. **Partición 80/20** con `train_test_split`. `random_state=RANDOM_STATE` hace la división reproducible y `stratify=X["smoker"]` conserva la misma proporción de fumadores en ambos conjuntos (importante porque `smoker` es la variable más influyente y está desbalanceada: unos 1064 de 1338 no fuman).
2. **Numéricas** (`num_f`): `SimpleImputer(strategy="median")` rellenaría nulos con la mediana (aquí no hay, es una medida preventiva) y `StandardScaler` estandariza cada columna a media 0 y desviación 1, de modo que los coeficientes sean comparables entre sí.
3. **Binarias** (`bin_f`): `OneHotEncoder(drop="if_binary")` crea **una sola** columna 0/1 por variable (`sex_male`, `smoker_yes`).
4. **Región** (`cat_f`): `OneHotEncoder(drop="first")` crea una columna por categoría y descarta la primera (`northeast`), que queda como categoría de referencia. Así se evita la **trampa de la variable dummy** (multicolinealidad perfecta).
5. **`fit_transform` solo sobre entrenamiento y `transform` sobre prueba:** el preprocesador aprende medias, desviaciones y categorías únicamente del conjunto de entrenamiento. Esto evita la **fuga de datos** (*data leakage*) desde la prueba.

Resultado: 8 columnas: `num__age`, `num__bmi`, `num__children`, `bin__sex_male`, `bin__smoker_yes`, `cat__region_northwest`, `cat__region_southeast`, `cat__region_southwest`.

[6]:
```
# X: todas las columnas menos la objetivo (axis por defecto al usar columns=)
X = df.drop(columns="charges")
# y: la variable que queremos predecir
y = df["charges"]

# Divide en entrenamiento (80 %) y prueba (20 %)
X_train, X_test, y_train, y_test = train_test_split(
    # Datos a dividir: predictoras y objetivo
    X, y,
    # test_size=0.2 -> 20 % para prueba; random_state -> división reproducible;
    # stratify=X["smoker"] -> mantiene la proporción de fumadores en ambos␣
 ↪conjuntos
```

```
    test_size=0.2, random_state=RANDOM_STATE, stratify=X["smoker"])

# Agrupa las columnas por tipo: numéricas, binarias y categórica multiclase
num_f, bin_f, cat_f = ["age", "bmi", "children"], ["sex", "smoker"], ["region"]

# ColumnTransformer: aplica una transformación distinta a cada grupo de columnas
preprocessor = ColumnTransformer([
    # Grupo "num": mini-pipeline de dos pasos aplicado a num_f
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),   # 1) rellena␣
 ↪nulos con la mediana
                      ("sc", StandardScaler())]), num_f),          # 2)␣
 ↪estandariza (media 0, desv. 1)
    # Grupo "bin": una columna 0/1 por variable binaria (elimina la columna␣
 ↪redundante)
    ("bin", OneHotEncoder(drop="if_binary"), bin_f),
    # Grupo "cat": columnas 0/1 por región, descartando la primera categoría␣
 ↪como referencia
    ("cat", OneHotEncoder(drop="first"), cat_f),
])

# Aprende los parámetros (medias, desviaciones, categorías) con train y los␣
 ↪aplica a train
X_train_p = preprocessor.fit_transform(X_train)
# Aplica a test los MISMOS parámetros aprendidos en train (sin volver a ajustar)
X_test_p = preprocessor.transform(X_test)
# Obtiene los nombres de las columnas resultantes (con prefijo num__, bin__,␣
 ↪cat__)
feat_names = preprocessor.get_feature_names_out()
# Convierte la matriz transformada de entrenamiento en DataFrame con nombres de␣
 ↪columna
X_train_df = pd.DataFrame(X_train_p, columns=feat_names)
# Muestra las primeras 5 filas ya transformadas
X_train_df.head()
```

[6]:

	num__age	num__bmi	num__children	bin__sex_male	bin__smoker_yes	\
0	0.209157	-0.959818	0.769598	0.0	0.0	
1	1.775700	1.370265	-0.076801	1.0	0.0	
2	0.992428	-0.220931	-0.923201	1.0	0.0	
3	-0.146876	0.882057	-0.923201	1.0	0.0	
4	-1.499800	1.225611	-0.923201	0.0	0.0	

	cat__region_northwest	cat__region_southeast	cat__region_southwest
0	1.0	0.0	0.0
1	0.0	1.0	0.0
2	0.0	1.0	0.0
3	0.0	1.0	0.0

4	0.0	1.0	0.0

0.9 Paso 7 — Selección de atributos

Qué hace este paso del pipeline: evalúa qué tan importante es cada variable para predecir `charges` usando **cuatro técnicas de naturaleza distinta**, todas calculadas solo con datos de entrenamiento. El objetivo es contrastar criterios, no descartar variables todavía.

Variables

Variable	Tipo	Descripción
`skb`	`SelectKBest` ajustado	Puntajes F de cada variable (`skb.scores_`); con `k="all"` no descarta ninguna
`mi`	arreglo	Información mutua de cada variable con `charges`
`rfe`	`RFE` ajustado	Indica con `rfe.support_` cuáles 5 variables conserva la eliminación recursiva
`lasso`	`Lasso` ajustado	Modelo con penalización L1; sus coeficientes (`lasso.coef_`) miden la importancia
`sel`	`DataFrame`	Tabla comparativa (una fila por variable, una columna por técnica)

Las cuatro técnicas

#	Técnica	Tipo	Idea
1	Prueba F (`f_regression`)	Filtro	Mide la relación **lineal** entre cada variable y el objetivo; mayor F = más relación
2	Información mutua	Filtro	Mide cualquier dependencia (incluso **no lineal**); mayor valor = más información sobre `charges`
3	RFE con `LinearRegression`	Wrapper	Entrena el modelo y elimina la variable menos importante, repetidamente, hasta quedar con 5

#	Técnica	Tipo	Idea
4	Lasso (`alpha=10`)	Embebido	La penalización L1 encoge coeficientes; los poco útiles tienden a 0

Lectura de resultados

- `smoker_yes` domina de lejos: F 1632 y coeficiente Lasso 23,682, y además RFE la conserva.
- `age` y `bmi` le siguen en importancia y también son conservadas por RFE.
- `children`, `sex_male` y `region_northwest` aparecen como las menos influyentes en la prueba F.
- La información mutua asigna el valor más alto a `age` (1.38), lo que sugiere una relación con `charges` que no es puramente lineal; por eso los rankings no coinciden entre técnicas.
- Con `alpha=10` Lasso no lleva ningún coeficiente exactamente a 0.

  Este paso es **informativo**: el modelo final de los pasos siguientes se entrena con las 8 variables.

[7]:
```
# 1. Filtro: F-test
# SelectKBest con k="all" calcula el puntaje F de cada variable sin descartar␣
 ↪ninguna; se ajusta solo con train
skb = SelectKBest(f_regression, k="all").fit(X_train_df, y_train)
# 2. Filtro: información mutua
# Mide la dependencia (lineal o no) entre cada variable y charges; random_state␣
 ↪fija el resultado
mi = mutual_info_regression(X_train_df, y_train, random_state=RANDOM_STATE)
# 3. Wrapper: RFE
# Entrena una regresión lineal y elimina variables una a una hasta conservar␣
 ↪las 5 mejores
rfe = RFE(LinearRegression(), n_features_to_select=5).fit(X_train_df, y_train)
# 4. Embebido: Lasso
# Regresión con penalización L1 (alpha=10 controla la fuerza de la penalización)
lasso = Lasso(alpha=10).fit(X_train_df, y_train)

# Reúne los 4 resultados en una tabla; el índice son los nombres de las␣
 ↪variables
sel = pd.DataFrame({"F": skb.scores_,               # puntaje F
                    "MI": mi,                        # información mutua
                    "RFE": rfe.support_,             # True = la variable fue␣
 ↪seleccionada por RFE
                    "Lasso_coef": lasso.coef_},      # coeficiente de Lasso
                   index=feat_names).round(3)        # redondea a 3 decimales
# Muestra la tabla ordenada de mayor a menor puntaje F
display(sel.sort_values("F", ascending=False))
```

	F	MI	RFE	Lasso_coef
bin__smoker_yes	1632.224	0.361	True	23682.390

```
num__age                97.155  1.381   True    3585.888
num__bmi                35.791  0.105   True    1991.156
num__children            6.071  0.138  False     729.098
cat__region_southeast    5.920  0.020   True   -1017.423
cat__region_southwest    2.585  0.012   True    -863.466
bin__sex_male            1.467  0.139  False    -215.058
cat__region_northwest    0.875  0.043  False    -197.388
```

0.10 Paso 8 — Validación cruzada (K-Fold)

Qué hace este paso del pipeline: estima de forma más confiable qué tan bien generalizará el modelo, usando **validación cruzada de 5 particiones** sobre el conjunto de entrenamiento. Así no se depende de una única división de los datos.

Variables

Variable	Tipo	Descripción
`pipe`	`Pipeline`	Preprocesador + `LinearRegression` encadenados en un solo objeto
`kf`	`KFold`	Esquema de validación: 5 bloques, con mezcla previa y semilla fija
`r2`	arreglo (5 valores)	R^2 obtenido en cada partición
`rmse`	arreglo (5 valores)	RMSE (raíz del error cuadrático medio) obtenido en cada partición

Proceso

1. Se arma `pipe` con el preprocesador **dentro**: en cada partición, el escalado y la codificación se reajustan solo con los datos de entrenamiento de esa partición, lo que evita fuga de datos.
2. `KFold(n_splits=5, shuffle=True)`: los datos se barajan y se parten en 5 bloques; cada bloque sirve una vez como validación mientras los otros 4 entrenan.
3. `cross_val_score` devuelve una métrica por bloque. Para el RMSE se usa `neg_root_mean_squared_error` (sklearn maximiza métricas, por eso devuelve el error en negativo) y se multiplica por -1.
4. Se reporta **media ± desviación estándar**: la media indica el desempeño típico y la desviación qué tanto varía entre particiones.

Resultado: R^2 CV $= 0.722 \pm 0.036$ y RMSE CV $= 6{,}297 \pm 629$. El modelo explica en promedio cerca del 72 % de la variabilidad de `charges` con un error típico de unos 6,300 en las mismas unidades de la variable.

[8]:
```
# Pipeline completo: primero el preprocesador (escalado + codificación), luego la regresión lineal
pipe = Pipeline([("prep", preprocessor), ("model", LinearRegression())])
# Validación cruzada de 5 bloques: baraja los datos (shuffle) con semilla fija para reproducibilidad
```

```
kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

# R² en cada bloque (usa X_train SIN transformar: el pipeline transforma␣
 ↪internamente)
r2 = cross_val_score(pipe, X_train, y_train, cv=kf, scoring="r2")
# RMSE en cada bloque; sklearn lo devuelve negativo, el signo "-" lo vuelve␣
 ↪positivo
rmse = -cross_val_score(pipe, X_train, y_train, cv=kf,␣
 ↪scoring="neg_root_mean_squared_error")
# Imprime R² promedio ± desviación estándar con 3 decimales
print(f"R² CV: {r2.mean():.3f} ± {r2.std():.3f}")
# Imprime RMSE promedio ± desviación estándar, con separador de miles y sin␣
 ↪decimales
print(f"RMSE CV: {rmse.mean():,.0f} ± {rmse.std():,.0f}")
```

```
R² CV: 0.722 ± 0.036
RMSE CV: 6,297 ± 629
```

0.11 Paso 9 — Entrenamiento del modelo

Qué hace este paso del pipeline: ajusta el modelo final con **todo el conjunto de entrenamiento** y extrae el intercepto y los coeficientes para interpretar el efecto de cada variable.

Variables

Variable	Tipo	Descripción
`pipe`	`Pipeline`	Tras `fit`, queda entrenado (aprende parámetros del preprocesador y del modelo)
`coefs`	`Series`	Coeficiente de regresión de cada variable, indexado por `feat_names`
`intercept_`	`float`	Valor base de la predicción cuando todas las variables valen 0

Modelo: regresión lineal múltiple,

$$\texttt{charges} = \texttt{intercepto} + \cdot\texttt{age} + \cdot\texttt{bmi} + \cdot\texttt{children} + \cdot\texttt{sex_male} + \cdot\texttt{smoker_yes} + \cdot\texttt{region_northwest} + \cdot\texttt{region_southeast} + \cdot\texttt{region_southwest}$$

Cómo interpretar los coeficientes

- Las variables numéricas están **estandarizadas**: su coeficiente es el cambio esperado en `charges` por cada aumento de **1 desviación estándar** de esa variable, manteniendo las demás constantes.
- Las variables dummy (0/1) indican la diferencia esperada respecto de la categoría de referencia (mujer, no fumador, región `northeast`).

- El **intercepto (9,284)** es la predicción para una mujer no fumadora de la región `northeast` con edad, IMC e hijos en el promedio.

Resultados: `smoker_yes` (+23,750) tiene por mucho el mayor efecto, seguido de `age` (+3,594) y `bmi` (+2,013). Las regiones `southeast` y `southwest` tienen coeficientes negativos frente a `northeast`.

[9]:
```
# Entrena el pipeline completo con los datos de entrenamiento (ajusta el␣
 ↪preprocesador y luego la regresión)
pipe.fit(X_train, y_train)
# Extrae los coeficientes del modelo (paso "model") y los asocia con el nombre␣
 ↪de cada variable
coefs = pd.Series(pipe.named_steps["model"].coef_, index=feat_names)
# Imprime el intercepto (término independiente) de la regresión
print("Intercepto:", pipe.named_steps["model"].intercept_)
# Muestra los coeficientes ordenados por magnitud (key=abs ordena por valor␣
 ↪absoluto, de mayor a menor)
display(coefs.sort_values(key=abs, ascending=False))
```

```
Intercepto: 9283.851527009758
```

```
bin__smoker_yes          23750.032145
num__age                  3594.126588
num__bmi                  2012.606363
cat__region_southeast    -1193.734426
cat__region_southwest    -1031.822157
num__children              739.762046
cat__region_northwest     -361.196690
bin__sex_male             -257.961317
dtype: float64
```

0.12 Paso 10 — Evaluación en datos no vistos y propuesta de mejora

Qué hace este paso del pipeline: mide el desempeño del modelo sobre el **conjunto de prueba** (datos que nunca usó para entrenar), compara contra una variante que transforma el objetivo con logaritmo y genera gráficos de diagnóstico.

Variables y funciones

Elemento	Descripción
`evaluar(modelo, nombre)`	Función que predice sobre `X_test` y devuelve un diccionario con MAE, RMSE, R^2 y MAPE
`pred`	Predicciones del modelo sobre `X_test`
`res`	Lista de diccionarios con las métricas de cada modelo evaluado
`pipe_log`	`TransformedTargetRegressor`: entrena con `log1p(charges)` y devuelve la predicción en escala original con `expm1`
`fig, ax`	Figura y ejes de los dos gráficos de diagnóstico

Métricas

Métrica	Qué mide	Mejor si…
MAE	Error absoluto promedio, en unidades de `charges`	es menor
RMSE	Raíz del error cuadrático medio; penaliza más los errores grandes	es menor
R^2	Proporción de la varianza de `charges` explicada	está más cerca de 1
MAPE	Error porcentual promedio	es menor

Mejora probada: como `charges` está sesgada a la derecha (Paso 4), se entrena otra variante con `log1p(charges)`. En la práctica el resultado es mixto:

Modelo	MAE	RMSE	R^2	MAPE
Lineal base	3,593	5,087	0.820	0.347
Lineal + log(charges)	3,976	7,906	0.566	0.256

El modelo con logaritmo reduce el **MAPE** (errores porcentuales menores en cargos bajos), pero empeora MAE, RMSE y R^2 en la escala original. Con estas métricas, el **modelo lineal base** es el mejor y es el que se usa en el Paso 11.

Gráficos de diagnóstico (modelo base)

- **Real vs predicho:** los puntos deberían alinearse con la diagonal roja (predicción perfecta).
- **Residuos vs predicho:** los residuos (real − predicho) deberían distribuirse alrededor de 0 sin patrón; una forma de abanico o curva indica varianza no constante o falta de linealidad.

  Nota: el R^2 en prueba (0.820) es mayor que el de validación cruzada (0.722). Esa diferencia puede deberse a que la partición de prueba resultó favorable; conviene tenerlo en cuenta al interpretar el resultado.

[10]:
```
# Función que evalúa cualquier modelo ya entrenado sobre el conjunto de prueba
def evaluar(modelo, nombre):
    # Predice charges para los datos de prueba (el modelo aplica su propio␣
 ↪preprocesamiento)
    pred = modelo.predict(X_test)
    # Devuelve un diccionario con el nombre del modelo y sus 4 métricas
    return {"Modelo": nombre,
            # Error absoluto medio
            "MAE": mean_absolute_error(y_test, pred),
            # Raíz del error cuadrático medio (se aplica sqrt al MSE)
            "RMSE": np.sqrt(mean_squared_error(y_test, pred)),
            # Coeficiente de determinación
            "R²": r2_score(y_test, pred),
            # Error porcentual absoluto medio
```

```
            "MAPE": mean_absolute_percentage_error(y_test, pred)}

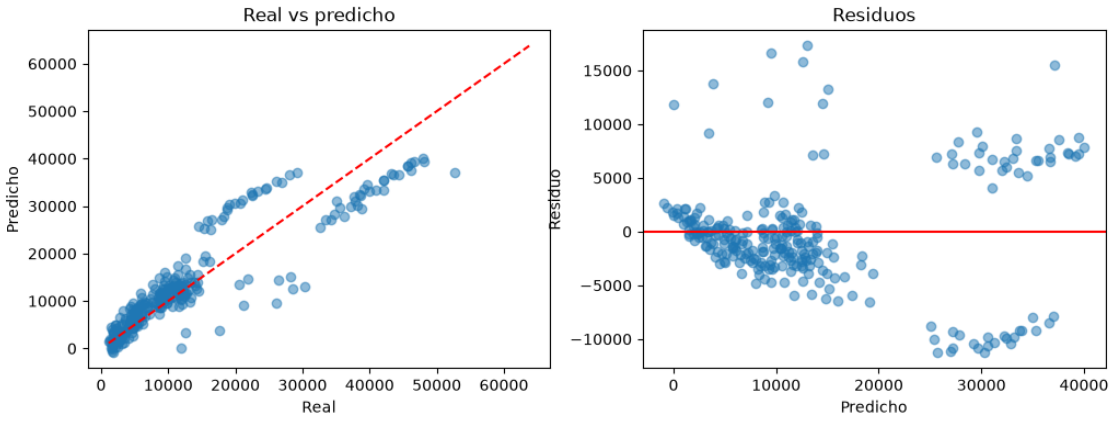
# Lista de resultados; inicia con la evaluación del modelo lineal base (pipe)
res = [evaluar(pipe, "Lineal base")]

# Mejora: log del objetivo (corrige el sesgo de charges)
# TransformedTargetRegressor: transforma y antes de entrenar y revierte la␣
 ↪transformación al predecir
from sklearn.compose import TransformedTargetRegressor
# Variante con log: regressor = mismo pipeline (preprocesador + regresión␣
 ↪lineal)
pipe_log = TransformedTargetRegressor(
    regressor=Pipeline([("prep", preprocessor), ("model", LinearRegression())]),
    # func: log(1+y) comprime la cola derecha; inverse_func: exp(x)-1 devuelve␣
 ↪a dólares
    func=np.log1p, inverse_func=np.expm1).fit(X_train, y_train)
# Agrega la evaluación de la variante con log a la lista de resultados
res.append(evaluar(pipe_log, "Lineal + log(charges)"))

# Convierte la lista de diccionarios en tabla comparativa y redondea a 3␣
 ↪decimales
display(pd.DataFrame(res).round(3))

# Gráficos de diagnóstico
# Predicciones del modelo base sobre el conjunto de prueba
pred = pipe.predict(X_test)
# Figura con 2 gráficos lado a lado
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
# Izquierda: dispersión real (X) vs predicho (Y)
ax[0].scatter(y_test, pred, alpha=.5)
# Diagonal roja punteada y = x: si el modelo fuera perfecto, todos los puntos␣
 ↪caerían sobre ella
ax[0].plot([y.min(), y.max()], [y.min(), y.max()], "r--")
# Etiquetas y título del gráfico izquierdo
ax[0].set(xlabel="Real", ylabel="Predicho", title="Real vs predicho")
# Derecha: residuos (real - predicho) frente al valor predicho
ax[1].scatter(pred, y_test - pred, alpha=.5)
# Línea roja horizontal en 0: referencia de error nulo
ax[1].axhline(0, color="r")
# Etiquetas y título del gráfico derecho
ax[1].set(xlabel="Predicho", ylabel="Residuo", title="Residuos")
# Muestra ambos gráficos
plt.show()
```

	Modelo	MAE	RMSE	R^2	MAPE
0	Lineal base	3592.793	5086.880	0.820	0.347
1	Lineal + log(charges)	3976.019	7905.986	0.566	0.256



0.13 Paso 11 — Predicción con nuevos datos

Qué hace este paso del pipeline: usa el modelo entrenado para estimar el costo del seguro de **personas que no están en el dataset**. Es el uso final del modelo.

Variables

Variable	Tipo	Descripción
`nuevos`	`DataFrame`	3 personas nuevas con las mismas 6 columnas predictoras que `X` (sin `charges`)
`charges_estimado`	columna nueva	Costo estimado por el modelo, redondeado a 2 decimales

Proceso

1. Se crea `nuevos` con los **mismos nombres de columna y categorías** que en el entrenamiento; es obligatorio, porque el preprocesador busca esas columnas por nombre.
2. `pipe.predict(nuevos)` aplica automáticamente el escalado y la codificación aprendidos en el Paso 6 y luego la regresión del Paso 9. No hay que transformar nada a mano.
3. El resultado se guarda en una columna nueva para ver cada perfil junto a su estimación.

Resultados

Perfil	Estimación
Mujer de 25 años, IMC 22.5, sin hijos, no fumadora, `southwest`	1,216
Hombre de 45 años, IMC 31.0, 2 hijos, fumador, `northeast`	34,923
Hombre de 60 años, IMC 35.2, 3 hijos, no fumador, `southeast`	15,834

El perfil fumador resulta el más costoso, coherente con el efecto dominante de `smoker_yes` visto en los pasos 7 y 9. Son estimaciones puntuales: el error típico en prueba (RMSE 5,087) indica la incertidumbre que debe asumirse.

[11]:
```
# Crea un DataFrame con 3 personas nuevas; cada clave es una columna y cada␣
 ↪lista tiene 3 valores (uno por persona)
nuevos = pd.DataFrame({
    # Edades de las 3 personas
    "age": [25, 45, 60],
    # Sexo de cada persona (mismas categorías que en el entrenamiento)
    "sex": ["female", "male", "male"],
    # Índice de masa corporal de cada persona
    "bmi": [22.5, 31.0, 35.2],
    # Número de hijos de cada persona
    "children": [0, 2, 3],
    # Condición de fumador de cada persona
    "smoker": ["no", "yes", "no"],
    # Región de residencia de cada persona
    "region": ["southwest", "northeast", "southeast"],
})
# Predice con el pipeline entrenado (preprocesa y calcula) y redondea a 2␣
 ↪decimales; guarda en nueva columna
nuevos["charges_estimado"] = pipe.predict(nuevos).round(2)
# Última expresión: muestra la tabla con los datos y su costo estimado
nuevos
```

[11]:

	age	sex	bmi	children	smoker	region	charges_estimado
0	25	female	22.5	0	no	southwest	1216.23
1	45	male	31.0	2	yes	northeast	34922.97
2	60	male	35.2	3	no	southeast	15833.71